University Help Desk System — Database Connectivity Briefing

Prepared for project defense: "here is the exact code that connects this feature to the database and how it runs end to end."

Project root referenced below: `university-help-desk/` (all paths are relative to it).

1. Tech Stack & Connection Configuration

Stack: Spring Boot **3.3.4** (`pom.xml` line 10) on **Java 21**, with:

- `spring-boot-starter-data-jpa` — Spring Data JPA, backed by **Hibernate** as the JPA provider (this is what turns Java objects into SQL).
- `com.microsoft.sqlserver:mssql-jdbc` (`pom.xml` lines 55–59), scope `runtime` — the actual JDBC driver class that opens a TCP connection to SQL Server and sends T-SQL over the wire. The pom comment explicitly says: *"Microsoft SQL Server JDBC driver (swapped from mysql-connector-j so the team can use SQL Server Management Studio, per the course's required tooling)"*. **This is important to say out loud to the lecturer: the database is Microsoft SQL Server, not MySQL**, even though some internal comments elsewhere in the code still reference MySQL habits (e.g. the `KbArticle.content` comment about `@Lob`/CLOB, and the `application.properties` comment "Unlike MySQL, SQL Server won't create the database for you").
- `spring-boot-starter-thymeleaf` + `thymeleaf-extras-springsecurity6` — server-rendered HTML views with `sec:` tags.
- `spring-boot-starter-security` — session-based login (not JWT).
- Lombok (`@Getter/@Setter/@Builder`) to cut boilerplate on entities.

Exact datasource/JPA configuration

`src/main/resources/application.properties`:

```
12 spring.datasource.url=${DB_URL:jdbc:sqlserver://localhost:1433;databaseName=helpdesk_db;e
ncrypt=true;trustServerCertificate=true}
13 spring.datasource.username=${DB_USERNAME:helpdesk_user}
14 spring.datasource.password=${DB_PASSWORD:YourStrongPassword123!}
15 spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
```

Line	What it means
12 `spring.datasource.url`	The JDBC connection string. `${DB_URL:default}` means: use environment variable `DB_URL` if set, otherwise fall back to the literal default shown — connecting to SQL Server on `localhost:1433`, database `helpdesk_db`, with TLS encryption on but the server certificate not validated (fine for a local dev box).
13 `spring.datasource.username`	Login used to authenticate to SQL Server — defaults to `helpdesk_user`.

Line	What it means
14 `spring.datasource.password`	Password for that login — defaults to `YourStrongPassword123!` (matches `SQLQuery2.sql`).
15 `spring.datasource.driver-class-name`	Forces the JDBC driver class explicitly to the Microsoft SQL Server driver, so Spring Boot doesn't try to auto-detect (or default to) a MySQL driver.

```
20 spring.jpa.hibernate.ddl-auto=${DDL_AUTO:update}
21 spring.jpa.show-sql=${SHOW_SQL:true}
22 spring.jpa.properties.hibernate.format_sql=true
25 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
35 spring.jpa.open-in-view=${OPEN_IN_VIEW:true}
```

Line	What it means
20 `ddl-auto=update`	On every app startup, Hibernate compares its in-memory model (built from `@Entity` classes) against the actual SQL Server schema and **automatically issues** `CREATE TABLE` **for missing tables,** `ALTER TABLE ... ADD COLUMN` **for missing columns**, etc. It never drops or renames columns. This is why the team never had to hand-write `CREATE TABLE` scripts — the 23 entity classes *are* the schema definition.
21 `show-sql=true`	Prints every generated SQL statement to the console/log — useful for proving to a lecturer, live, that a specific button click really issues a specific `INSERT`/`UPDATE`/`SELECT`.
25 `hibernate.dialect=SQLServerDialect`	Tells Hibernate to generate T-SQL syntax specific to SQL Server (e.g. `TOP` instead of `LIMIT`, `IDENTITY` columns) rather than generic or MySQL SQL.
35 `open-in-view=true`	Keeps the Hibernate session (and its DB connection) open for the *entire* HTTP request, through Thymeleaf rendering. The comment on lines 26–34 explains why: most entity associations (`Ticket.student`, `KbArticle.category`, etc.) are `FetchType.LAZY`, meaning the related row is not fetched until you actually call the getter (e.g. `${ticket.student.fullName}` inside a Thymeleaf template). If the session had already closed (open-in-view=false, Spring Boot's non-default), that lazy call throws `LazyInitializationException`. Keeping it open avoids adding `JOIN FETCH` to every query.

Also relevant: `spring.servlet.session.timeout=30m` (line 45, pure session config, not DB), and `app.seed-demo-data` (line 56, see Section 6).

`SQLQuery2.sql` — the one-time manual setup script a team member runs **once in SSMS** before the app can connect:

```
1 USE helpdesk_db;
2 ALTER LOGIN helpdesk_user WITH PASSWORD = 'YourStrongPassword123!';
3 ALTER ROLE db_owner ADD MEMBER helpdesk_user;
```

This assumes `helpdesk_db` and the `helpdesk_user` login already exist (created via SSMS or `sqlcmd` separately, per the `application.properties` line 7–8 comment: *"SQL Server won't create the database for you — it must already exist"*), then resets the login's password and grants it `db_owner` (full read/write/DDL rights on `helpdesk_db`) so Hibernate's `ddl-auto=update` can actually create/alter tables.

2. The 4-Layer Architecture (explained once)

Every feature in this app follows the same four layers:

```
Browser (Thymeleaf form/link)
| HTTP GET/POST
|
CONTROLLER (@Controller, @RequestMapping) — src/main/java/com/helpdesk/controller/
| calls a service method, passes plain Java objects (entities/DTOs)
|
SERVICE (interface + Impl, @Service, often @Transactional) — .../service/, .../service/impl/
| business rules, then calls repository methods
|
REPOSITORY (interface extends JpaRepository<Entity, Long>) — .../repository/
| Spring Data JPA generates the SQL — NO hand-written SQL or JDBC code anywhere in this
project
|
ENTITY (@Entity, @Table, @Column, @ManyToOne, ...) — .../entity/
| Hibernate ORM maps the Java class <-> a SQL Server table
|
Microsoft SQL Server (database "helpdesk_db")
```

What actually happens when you call `articleRepository.save(article)`:

1. `KbArticleRepository extends JpaRepository<KbArticle, Long>` — you never implement `save()` yourself. At startup, Spring Data JPA generates a runtime proxy class implementing this interface.

2. That proxy delegates to Hibernate's `EntityManager.persist(...)` (new row, no id yet) or `EntityManager.merge(...)` (row already has an id — an update).

3. Hibernate looks at the `@Entity`/`@Table(name="kb_articles")`/`@Column(...)` annotations on `KbArticle.java` to know exactly which table and which columns to target, and builds a parameterized SQL statement: `INSERT INTO kb_articles (title, content, category_id, author_id, status, view_count, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?)` (or the equivalent `UPDATE ... WHERE id = ?` if the entity already had an id).

4. Hibernate hands that statement + bound parameter values to the JDBC driver (`mssql-jdbc`), which sends it over the TCP connection (configured via `spring.datasource.url`) to SQL Server as a `PreparedStatement`.

5. SQL Server executes it, and (for `@GeneratedValue(strategy = GenerationType.IDENTITY)`, used on every entity's `id`) returns the generated identity value, which Hibernate sets back onto the Java object's `id` field.

What happens on `ticketRepository.findById(id)`: the generated proxy calls `EntityManager.find(Ticket.class, id)`, Hibernate builds `SELECT * FROM tickets WHERE id = ?` (using the `@Table`/`@Column` mapping again), sends it via JDBC, and maps each returned row's columns back onto a new `Ticket` Java object (its `@ManyToOne` fields like `student`/`category` are loaded as lazy proxies, not eagerly joined, per Section 1).

Custom queries (`@Query("SELECT t FROM Ticket t WHERE ...")`) use **JPQL** (Java Persistence Query Language) — it looks like SQL but references entity/field names (`Ticket`, `t.assignedStaff.id`) instead of

table/column names. Hibernate translates JPQL into the actual T-SQL sent to SQL Server. **Derived query methods** (e.g. `findByStudentIdOrderByCreatedAtDesc`) need no `@Query` at all — Spring Data JPA parses the method name itself (`findBy` + `StudentId` + `OrderBy` + `CreatedAtDesc`) and builds the equivalent JPQL/SQL automatically. This is the single most important thing to be able to explain to a lecturer: **there is no hand-written SQL/JDBC `Connection`/`Statement` code anywhere in this codebase** — every database operation goes through this Controller -> Service -> Repository (Spring Data JPA / Hibernate) -> Entity -> SQL Server pipeline.

3. Per-Feature Breakdown

3.1 Authentication & User Accounts (Login, Self-Registration, Admin-created Staff/Admin)

Entity: `User` (`entity/User.java`) -> table `users` (`@Table(name = "users")`, line 25), indexed on `role` (line 26). Columns: `full_name`, `university_id` (unique), `email` (unique), `password_hash` (BCrypt hash, never plaintext), `role` (enum stored as string: STUDENT/STAFF/ADMIN), `status` (enum: ACTIVE/DEACTIVATED), `created_at`/`updated_at` (auto-set via `@PrePersist`/`@PreUpdate`, lines 75–83).

Repository: `UserRepository` (`repository/UserRepository.java`):

```
Optional<User> findByEmail(String email);
boolean existsByEmail(String email);
boolean existsByUniversityId(String universityId);
```

All three are derived queries — no `@Query` needed; Spring Data JPA turns `findByEmail` into `SELECT * FROM users WHERE email = ?`.

Login (Spring Security, not a controller): `security/CustomUserDetailsService.java` implements `UserDetailsService.loadUserByUsername(email)` — called automatically by Spring Security's `DaoAuthenticationProvider` on every login POST:

```
User user = userRepository.findByEmail(email)
.orElseThrow(() -> new UsernameNotFoundException("No account found for email: " + email));
return new CustomUserDetails(user);
```

`security/CustomUserDetails.java` wraps the `User` entity to satisfy Spring Security's `UserDetails` contract (`getPassword()` returns `user.getPasswordHash()`, `getAuthorities()` returns `ROLE_<role>`, `isEnabled()`/`isAccountNonLocked()` check `AccountStatus`). `SecurityConfig.java` line 37 registers `BCryptPasswordEncoder` as the `PasswordEncoder` bean, so Spring Security BCrypt-compares the submitted password against `password_hash` — this comparison happens in memory, not in SQL. On success, `RoleBasedAuthenticationSuccessHandler` (`security/RoleBasedAuthenticationSuccessHandler.java`) redirects by role.

Self-registration: `AuthController.register()` (`controller/AuthController.java` lines 39–59) -> `UserService.registerStudent(form)` -> `UserServiceImpl.registerStudent()` (`service/impl/UserServiceImpl.java` lines 41–59):

```
if (userRepository.existsByEmail(form.getEmail())) throw new
DuplicateResourceException(...);
if (userRepository.existsByUniversityId(form.getUniversityId())) throw new
DuplicateResourceException(...);
User user = User.builder()...passwordHash(passwordEncoder.encode(form.getPassword())).role(R
ole.STUDENT)...build();
return userRepository.save(user);
```

Admin creates Staff/Admin: `AdminUserController.create()` (`controller/AdminUserController.java` lines 37–59) -> `UserServiceImpl.createStaffAccount()` (lines 62–88) generates a random temp password, BCrypt-encodes it, `userRepository.save(user)`, then writes an **audit log row** via `auditLogService.log(...)` (Section 3.9).

End-to-end trace — "student logs in":

1. Browser POSTs `email`/`password` to `/login` (form defined by `SecurityConfig.formLogin()`, lines 60–67 — note `usernameParameter("email")`).

2. Spring Security's filter chain intercepts this **before any** `@Controller` **method runs**.

3. `DaoAuthenticationProvider` calls `CustomUserDetailsService.loadUserByUsername(email)` (`security/CustomUserDetailsService.java` line 25).

4. That calls `userRepository.findByEmail(email)` -> Hibernate issues `SELECT * FROM users WHERE email = ?` to SQL Server.

5. Spring Security BCrypt-compares the posted password against the returned `password_hash`.

6. On success, `RoleBasedAuthenticationSuccessHandler.onAuthenticationSuccess()` redirects to `/student/dashboard`, `/staff/dashboard`, or `/admin/dashboard` based on the user's `ROLE_*` authority.

3.2 Ticket Management (Create / Comment / Status / Assignment / Escalation / Reopen)

Entities:

- `Ticket` -> table `tickets` (`entity/Ticket.java` line 22). FKs: `student_id` -> `users` (`@ManyToOne student`), `category_id` -> `ticket_categories` (`category`), `assigned_staff_id` -> `users`, nullable (`assignedStaff`). Also `ticket_ref` (unique human ref like `TCK-000123`), `status` (enum: OPEN/IN_PROGRESS/RESOLVED/CLOSED/ESCALATED/REOPENED), `priority` (LOW/MEDIUM/HIGH).
- `TicketComment` -> table `ticket_comments` (`entity/TicketComment.java`). FKs: `ticket_id` -> `tickets`, `author_id` -> `users`. Field `is_resolution_note` (boolean).
- `TicketAttachment` -> table `ticket_attachments`. FK `ticket_id` -> `tickets`; stores `file_name`/`file_path`.
- `TicketCategory` -> table `ticket_categories`, FK `pool_id` -> `job_pools` (nullable `JobPool pool`).

Repository: `TicketRepository` (`repository/TicketRepository.java`):

```
List<Ticket> findByStudentIdOrderByCreatedAtDesc(Long studentId); // derived query
List<Ticket> findAllByOrderByCreatedAtDesc(); // derived query
long countByStatus(TicketStatus status); // derived query

@Query("SELECT t FROM Ticket t WHERE t.assignedStaff.id = :staffId " +
"OR (t.assignedStaff IS NULL AND t.category.pool IN " +
" (SELECT p FROM JobPool p JOIN p.staffMembers s WHERE s.id = :staffId)) " +
"ORDER BY t.createdAt DESC")
List<Ticket> findQueueForStaff(@Param("staffId") Long staffId);
```

`findQueueForStaff` is the routing mechanism in practice: a staff member's queue = tickets already assigned to them, **plus** unassigned tickets whose category's `pool` (set on `TicketCategory.pool`, an Admin config field) they belong to (via the `pool_staff` join table on `JobPool.staffMembers`).

Service: `TicketService`/`TicketServiceImpl` (`service/impl/TicketServiceImpl.java`):

```
@Transactional
public Ticket createTicket(User student, TicketForm form) {
TicketCategory category = categoryRepository.findById(form.getCategoryId())...;
Ticket ticket = Ticket.builder().student(student).category(category)
.subject(form.getSubject()).description(form.getDescription())
.status(TicketStatus.OPEN).priority(Priority.MEDIUM).ticketRef("PENDING").build();
Ticket saved = ticketRepository.save(ticket);
saved.setTicketRef(String.format("TCK-%06d", saved.getId())); // needs the generated id
first
return saved;
}
```

`assignToSelf()`, `updateStatus()`, `addComment()`, `escalate()`, `reopen()` all: (1) `getById()` (a `SELECT`), (2) mutate the managed entity's fields directly (no explicit `save()` call needed — inside an open `@Transactional` Hibernate session, a "dirty" managed entity is auto-flushed to an `UPDATE` at commit — this is called the **dirty-checking mechanism**), (3) call `notificationService.notify(...)` to insert a `notifications` row, and (4) for `escalate()`, also `auditLogService.log(...)`.

Controllers:

- `TicketController` (student): `GET /student/tickets` (list+form), `POST /student/tickets` (create), `GET /student/tickets/{id}` (detail), `POST /student/tickets/{id}/reopen`, `POST /student/tickets/{id}/feedback`.
- `StaffTicketController`: `GET /staff/tickets` (queue), `GET /staff/tickets/{id}`, `POST /staff/tickets/{id}/assign`, `POST /staff/tickets/{id}/status`, `POST /staff/tickets/{id}/comment`, `POST /staff/tickets/{id}/escalate`.

End-to-end trace — "student submits a new ticket":

1. `POST /student/tickets` hits `TicketController.create()` (`controller/TicketController.java` lines 45–55).

2. It calls `ticketService.createTicket(principal.getUser(), form)`.

3. `TicketServiceImpl.createTicket()` (`service/impl/TicketServiceImpl.java` lines 38–55) first does `categoryRepository.findById(...)` -> `SELECT * FROM ticket_categories WHERE id = ?`.

4. Builds a new `Ticket` object and calls `ticketRepository.save(ticket)` -> Hibernate issues `INSERT INTO tickets (student_id, category_id, subject, description, status, priority, ticket_ref, created_at) VALUES (...)`, SQL Server returns the generated `IDENTITY` id.

5. Service sets the human-readable `ticket_ref` (`TCK-000123`) and, because the method is `@Transactional`, this field change is dirty-checked and flushed as an `UPDATE tickets SET ticket_ref = ? WHERE id = ?` when the transaction commits.

6. Controller redirects the browser to `/student/tickets/{id}`, which triggers `TicketController.detail()` -> another `SELECT` for the ticket plus `ticketService.commentsFor(id)` (`SELECT * FROM ticket_comments WHERE ticket_id = ? ORDER BY created_at`).

(Note: this project does not automatically apply a `RoutingRule` at creation time — routing only affects who sees the ticket later, via `findQueueForStaff`'s lookup of `category.pool`; see Section 3.9 for the separate `RoutingRule` admin-config feature.)

3.3 Knowledge Base + Chatbot

Entities: `KbArticle` -> table `kb_articles` (`entity/KbArticle.java`), FKs `category_id` -> `kb_categories`, `author_id` -> `users`; fields `title`, `content` (mapped `columnDefinition = "TEXT"`, deliberately *not* `@Lob` — the code comment explains Hibernate 6 would map `@Lob` to CLOB and then refuse to pass it to `LOWER()` in the search query), `status` (DRAFT/PUBLISHED), `view_count`. `KbCategory` -> table `kb_categories`.

Repository: `KbArticleRepository` (`repository/KbArticleRepository.java`):

```
@Query("SELECT a FROM KbArticle a WHERE a.status = :status " +
"AND (:keyword IS NULL OR :keyword = '' " +
" OR LOWER(a.title) LIKE LOWER(CONCAT('%', :keyword, '%')) " +
" OR LOWER(a.content) LIKE LOWER(CONCAT('%', :keyword, '%'))) " +
"ORDER BY a.viewCount DESC")
List<KbArticle> search(@Param("status") KbArticleStatus status, @Param("keyword") String
keyword);
```

Service (article CRUD): `KnowledgeBaseServiceImpl.saveArticle()` (`service/impl/KnowledgeBaseServiceImpl.java` lines 68–84) branches on whether `form.getId()` is set (update existing managed entity vs. build a new one), then `articleRepository.save(article)`. `getByIdAndIncrementViews()` (lines 40–45) fetches the article then `article.setViewCount(article.getViewCount() + 1)` — dirty-checked into an `UPDATE` on commit.

Service (chatbot): `ChatbotServiceImpl.respond()` (`service/impl/ChatbotServiceImpl.java`) — **there is no separate chat-log table**; the chatbot is a keyword search directly against `kb_articles`:

```
List<KbArticle> matches = articleRepository.search(KbArticleStatus.PUBLISHED, cleaned);
if (matches.isEmpty() && words.length > 1) {
// fall back to the single longest word in the message
matches = articleRepository.search(KbArticleStatus.PUBLISHED, longestWord);
}
```

Controllers:

- `KnowledgeBaseController` (student-facing): `GET /student/kb?keyword=` (search), `GET /student/kb/{id}` (view + increments `view_count`), `POST /student/kb/chatbot` (AJAX, `@ResponseBody`, tracks failed-attempt count in the HTTP session to eventually suggest raising a ticket — that counter is session state, not a DB write).
- `StaffKbController`: `GET /staff/kb` (list+edit form), `POST /staff/kb` (create/update), `POST /staff/kb/categories` (new `KbCategory`), `POST /staff/kb/{id}/delete`.

End-to-end trace — "staff creates a KB article":

1. `POST /staff/kb` -> `StaffKbController.save()` (`controller/StaffKbController.java` lines 43–53).

2. Calls `knowledgeBaseService.saveArticle(form, principal.getUser())`.

3. `KnowledgeBaseServiceImpl.saveArticle()` does `categoryRepository.findById(...)` (`SELECT` on `kb_categories`), builds a new `KbArticle.builder().author(author).viewCount(0).build()` since `form.getId()` is null, sets title/content/category/status.

4. `articleRepository.save(article)` -> `INSERT INTO kb_articles (title, content, category_id, author_id, status, view_count, created_at) VALUES (...)`.

5. Redirect to `/staff/kb`, which re-runs `articleRepository.findAllByOrderByCreatedAtDesc()` and now shows the new row.

3.4 Lost & Found

Entities: `LostFoundItem` -> table `lost_found_items` (`entity/LostFoundItem.java`), FK `reporter_id` -> `users`; fields `item_type` (LOST/FOUND), `description`, `location`, `item_date`, `status` (ACTIVE/CLAIMED). `LostFoundClaim` -> table `lost_found_claims`, FKs `item_id` -> `lost_found_items`, `claimant_id` -> `users`; unique constraint `(item_id, claimant_id)` prevents duplicate claims (line 19–21); `status` (PENDING/VERIFIED/REJECTED).

Repository: `LostFoundItemRepository` — `search()` is a `@Query` filtering by status/type/keyword against `description`/`location` with `LOWER(...) LIKE ...`. `LostFoundClaimRepository` — `findByItemIdAndClaimantId` (derived, used for the duplicate-claim check), `findByStatusOrderByCreatedAtAsc`.

Service: `LostFoundServiceImpl.claimItem()` (`service/impl/LostFoundServiceImpl.java` lines 60–80):

```
claimRepository.findByItemIdAndClaimantId(itemId, claimant.getId())
.ifPresent(existing -> { throw new IllegalStateException("You have already submitted a
claim..."); });
LostFoundClaim claim =
LostFoundClaim.builder().item(item).claimant(claimant).status(ClaimStatus.PENDING).build();
LostFoundClaim saved = claimRepository.save(claim);
notificationService.notify(item.getReporter(), "LOST_FOUND_CLAIM", ...);
```

`decideClaim()` (lines 93–113): sets claim status to VERIFIED/REJECTED, on approval also sets `item.setStatus(ItemStatus.CLAIMED)` (dirty-checked `UPDATE` on `lost_found_items`), writes an audit log entry, and notifies the claimant.

Controllers: `LostFoundController` (student): `GET /student/lost-found` (search+list), `POST /student/lost-found/report`, `GET /student/lost-found/{id}`, `POST /student/lost-found/{id}/claim`. `AdminLostFoundController`: `GET /admin/lost-found` (pending claims queue), `POST /admin/lost-found/{claimId}/approve`, `POST /admin/lost-found/{claimId}/reject`.

End-to-end trace — "student reports a lost item":

1. `POST /student/lost-found/report` -> `LostFoundController.report()` (`controller/LostFoundController.java` lines 38–49).

2. Calls `lostFoundService.reportItem(principal.getUser(), form)`.

3. `LostFoundServiceImpl.reportItem()` (lines 35–46) builds a `LostFoundItem` with `status(ItemStatus.ACTIVE)` and calls `itemRepository.save(item)` -> `INSERT INTO lost_found_items (reporter_id, item_type, description, location, item_date, status, created_at) VALUES (...)`.

4. Redirect to `/student/lost-found?reported=1`, which re-queries `itemRepository.search(...)` and shows the new listing.

3.5 Semester Registration

Entities: `Semester` -> table `semesters` (`registration_open` flag). `Course` -> table `courses` (`code` unique, `credits`). `CourseOffering` -> table `course_offerings`, FKs `semester_id` -> `semesters`, `course_id` -> `courses`, unique `(semester_id, course_id)`. `Registration` -> table `registrations`, FKs `student_id` -> `users`, `semester_id` -> `semesters`, unique `(student_id, semester_id)`; `courseOfferings` is `@ManyToMany` via join table `registration_courses` (`registration_id`, `course_offering_id`).

Repository: `SemesterRepository.findByRegistrationOpenTrue()`, `RegistrationRepository.findByStudentIdAndSemesterId(...)`, `CourseOfferingRepository.findBySemesterIdOrderByCourseCodeAsc(...)` — all derived queries.

Service: `RegistrationServiceImpl.register()` (`service/impl/RegistrationServiceImpl.java` lines 118–156) — the most business-rule-heavy method in the app:

```
if (!semester.isRegistrationOpen()) throw new BusinessRuleException("Registration is closed
for " + ...);
...
for (Long offeringId : offeringIds) {
CourseOffering offering = offeringRepository.findById(offeringId)...;
offerings.add(offering);
totalCredits += offering.getCourse().getCredits();
}
if (totalCredits > MAX_CREDITS_PER_SEMESTER) throw new BusinessRuleException(...); //
18-credit cap
Registration registration =
registrationRepository.findByStudentIdAndSemesterId(student.getId(), semesterId)
.orElse(Registration.builder().student(student).semester(semester).build());
registration.setCourseOfferings(offerings);
registration.setStatus(RegistrationStatus.CONFIRMED);
Registration saved = registrationRepository.save(registration);
notificationService.notify(student, "REGISTRATION_CONFIRMED", ...);
```

Note (per the class-level Javadoc): there is **no payment step** — this is a documented Phase-0 scope decision.

Controllers: `RegistrationController` (student): `GET /student/registration`, `POST /student/registration`. `AdminRegistrationConfigController`: `POST /admin/registration-config/semesters`, `/courses`, `/offerings`.

End-to-end trace — "student registers for courses":

1. `POST /student/registration` -> `RegistrationController.register()` (`controller/RegistrationController.java` lines 42–53).

2. Calls `registrationService.register(principal.getUser(), semesterId, offeringIds)`.

3. `RegistrationServiceImpl.register()` runs the credit-limit business rule (above), then `registrationRepository.save(registration)` — if new, `INSERT INTO registrations (student_id, semester_id, status, created_at) VALUES (...)` plus one `INSERT INTO registration_courses (registration_id, course_offering_id)` row per selected offering (Hibernate manages the join table automatically because of `@ManyToMany`/`@JoinTable`); if updating an existing registration, the join rows are replaced.

4. `notificationService.notify(...)` inserts a row into `notifications`.

5. Redirect back to `/student/registration` with a flash "Registration confirmed." message.

3.6 Notifications

Entity: `Notification` -> table `notifications` (`entity/Notification.java`), FK `recipient_id` -> `users`; fields `type`, `message`, `related_entity_type`/`related_entity_id` (a loose polymorphic reference so a notification can point back at *any* source table without a hard FK to each one), `is_read`.

Repository: `NotificationRepository`:

```
List<Notification> findTop20ByRecipientIdOrderByCreatedAtDesc(Long recipientId);
long countByRecipientIdAndReadFalse(Long recipientId);
```

Service: `NotificationServiceImpl.notify()` (`service/impl/NotificationServiceImpl.java` lines 29–40) is called **from every other feature's service** (Ticket, LostFound, Registration) whenever something notification-worthy happens — as the class Javadoc states, *"There's no message queue — this is a simple, synchronous INSERT."* `markAllRead()` fetches the top 20 and flips `read=true` on each (dirty-checked, one `UPDATE` per row on commit).

How it reaches every page: `GlobalModelAttributesAdvice` (`controller/GlobalModelAttributesAdvice.java`), a `@ControllerAdvice` with `@ModelAttribute`, runs before *every* controller method and calls `notificationService.unreadCount(userId)` + `recentForUser(userId)` so the notification bell/count is always populated in the shared layout — this means on **every single page load**, two extra `SELECT`s run against `notifications`.

End-to-end trace — "a ticket status change notifies the student": covered inside Section 3.2's trace: `TicketServiceImpl.notifyStudent()` (private helper, line 153) calls `notificationService.notify(ticket.getStudent(), "TICKET_STATUS", message, "Ticket", ticket.getId())`, which is `notificationRepository.save(...)` -> `INSERT INTO notifications (recipient_id, type, message, related_entity_type, related_entity_id, is_read, created_at) VALUES (...)`.

3.7 Feedback

Entity: `Feedback` -> table `feedback` (`entity/Feedback.java`), `@OneToOne` FK `ticket_id` -> `tickets` (`unique = true`, so one feedback row per ticket max), FK `student_id` -> `users`, FK `staff_id` -> `users`; `rating` (1–5, `@Min`/`@Max`), optional `comment`.

Repository: `FeedbackRepository.findByTicketId(Long)` (derived; used both to check "already submitted" and to display it), `findByStaffId(Long)`.

Service: `FeedbackServiceImpl.submitFeedback()` (`service/impl/FeedbackServiceImpl.java` lines 26–46):

```
if (ticket.getStatus() != TicketStatus.RESOLVED && ticket.getStatus() !=
TicketStatus.CLOSED)
throw new BusinessRuleException("Feedback can only be submitted once a ticket is Resolved or
Closed.");
if (feedbackRepository.findByTicketId(ticket.getId()).isPresent())
throw new BusinessRuleException("Feedback has already been submitted for this ticket.");
if (ticket.getAssignedStaff() == null)
throw new BusinessRuleException("This ticket has no assigned staff member to rate.");
Feedback feedback =
Feedback.builder().ticket(ticket).student(student).staff(ticket.getAssignedStaff())
.rating(rating).comment(comment).build();
return feedbackRepository.save(feedback);
```

Controller: `TicketController.submitFeedback()` — `POST /student/tickets/{id}/feedback` (`controller/TicketController.java` lines 79–93), reachable only after a ticket is Resolved/Closed.

End-to-end trace: POST -> `TicketController.submitFeedback()` -> `ticketService.getById(id)` (SELECT tickets) -> `feedbackService.submitFeedback(...)` runs the 3 business checks above -> `feedbackRepository.save(feedback)` -> `INSERT INTO feedback (ticket_id, student_id, staff_id, rating, comment, created_at) VALUES (...)` -> redirect back to the ticket detail page, which now also re-fetches `feedbackService.findByTicketId(id)` to display the submitted rating.

3.8 Reporting

No dedicated entity/table — this feature is **read-only aggregation** over `tickets` and `feedback`. `ReportingServiceImpl.generateSummary()` (`service/impl/ReportingServiceImpl.java`):

```
List<Ticket> allTickets = ticketRepository.findAll(); // SELECT * FROM tickets
for (TicketStatus status : TicketStatus.values())
countsByStatus.put(status.name(), ticketRepository.countByStatus(status)); // one SELECT
COUNT(*) per status
double avgResolutionHours = allTickets.stream()
.filter(t -> t.getResolvedAt() != null)
.mapToLong(t -> Duration.between(t.getCreatedAt(), t.getResolvedAt()).toMinutes())
.average().orElse(0) / 60.0; // computed in Java, not SQL
List<Feedback> feedbackList = feedbackRepository.findAll(); // SELECT * FROM feedback
double avgRating = feedbackList.stream().mapToInt(Feedback::getRating).average().orElse(0);
```

`countByStatus` is a Spring Data JPA derived query returning `SELECT COUNT(*) FROM tickets WHERE status = ?`; the resolution-time and rating averages are computed in application code after the rows are pulled back, not via SQL `AVG()`.

Controllers: `ReportController` — `GET /reports` (shared by Staff+Admin, deliberately outside `/staff` or `/admin` path prefixes so `SecurityConfig`'s `hasAnyRole("STAFF","ADMIN")` rule on `/reports` applies). `AdminDashboardController` — `GET /admin/dashboard` calls the same `reportingService.generateSummary()` plus `userService.findAllStaffAndAdmins().size()` and `lostFoundService.pendingClaims().size()` for dashboard tiles.

3.9 Admin Config (Routing Rules, Job Pools, Ticket Categories, Audit Log)

Entities:

- `TicketCategory` -> table `ticket_categories`, optional FK `pool_id` -> `job_pools`.
- `JobPool` -> table `job_pools`; `staffMembers` is `@ManyToMany` via join table `pool_staff` (`pool_id`, `user_id`).

- `RoutingRule` -> table `routing_rules`, FKs `category_id` -> `ticket_categories`, `target_pool_id` -> `job_pools`; **unique constraint** `(category_id, priority)` (`entity/RoutingRule.java` lines 21–23) — this is the database-level enforcement of "no two rules for the same category+priority", backing up the equivalent application-level check.
- `AuditLog` -> table `audit_logs`, FK `actor_id` -> `users`; fields `action`, `target_entity`, `target_id`, `details`.

Repositories: `RoutingRuleRepository.existsByCategoryAndPriority(category, priority)` (used for the friendly duplicate-rule error), `JobPoolRepository.existsByNameIgnoreCase`, `AuditLogRepository.findTop100ByOrderByCreatedAtDesc()`.

Service — routing rules: `AdminConfigServiceImpl.createRoutingRule()` (`service/impl/AdminConfigServiceImpl.java` lines 115–138):

```
if (routingRuleRepository.existsByCategoryAndPriority(category, form.getPriority()))
throw new BusinessRuleException("A routing rule already exists for " + category.getName() +
" + " + form.getPriority() + "...");
RoutingRule rule = RoutingRule.builder().category(category).priority(form.getPriority()).tar
getPool(pool).build();
RoutingRule saved = routingRuleRepository.save(rule);
auditLogService.log(actingAdmin, "CREATE_ROUTING_RULE", "RoutingRule", saved.getId(),
category.getName() + "/" + form.getPriority() + " -> " + pool.getName());
```

Service — job pools & staff assignment: `assignStaffToPool()`/`removeStaffFromPool()` mutate `pool.getStaffMembers()` (a `Set<User>`), which Hibernate persists as `INSERT`/`DELETE` rows in the `pool_staff` join table on commit.

Service — audit log: `AuditLogService` (`service/AuditLogService.java`, a plain concrete `@Service`, no interface) is called from `UserServiceImpl` (staff creation, activate/deactivate), `TicketServiceImpl` (escalate), `LostFoundServiceImpl` (claim decisions), and `AdminConfigServiceImpl` (every config change):

```
public void log(User actor, String action, String targetEntity, Long targetId, String
details) {
AuditLog entry = AuditLog.builder().actor(actor).action(action).targetEntity(targetEntity)
.targetId(targetId).details(details).build();
auditLogRepository.save(entry);
}
```

Controllers: `AdminConfigController` — `GET /admin/config`, `POST /admin/config/categories`, `/pools`, `/pools/{poolId}/staff`, `/pools/{poolId}/staff/{staffUserId}/remove`, `/routing-rules`, `/routing-rules/{id}/delete`. `AdminAuditLogController` — `GET /admin/audit-log`.

End-to-end trace — "admin creates a routing rule":

1. `POST /admin/config/routing-rules` -> `AdminConfigController.createRule()` (`controller/AdminConfigController.java` lines 91–104).

2. Calls `adminConfigService.createRoutingRule(form, principal.getUser())`.

3. `AdminConfigServiceImpl.createRoutingRule()` does two lookups (`categoryRepository.findById`, `poolRepository.findById` — two `SELECT`s), the `existsByCategoryAndPriority` duplicate check (a third `SELECT ... WHERE category_id = ? AND priority = ?`), then `routingRuleRepository.save(rule)` -> `INSERT INTO routing_rules (category_id, priority, target_pool_id, created_at) VALUES (...)` (rejected by SQL Server's unique constraint as a safety net even if the application check were bypassed).

4. `auditLogService.log(...)` -> `INSERT INTO audit_logs (actor_id, action, target_entity, target_id, details, created_at) VALUES (...)`.

5. Redirect to `/admin/config`, which re-runs `adminConfigService.listRoutingRules()` and shows the new rule.

4. Full Request-Flow Diagrams (text form)

4.1 Student Login

1. Browser -> `POST /login` with `email`/`password` (form built by `SecurityConfig.formLogin()`, `config/SecurityConfig.java` lines 60–67).

2. Spring Security's `DaoAuthenticationProvider` (auto-configured because exactly one `UserDetailsService` bean and one `PasswordEncoder` bean exist — comment at `SecurityConfig.java` lines 15–23) calls `CustomUserDetailsService.loadUserByUsername(email)` — `security/CustomUserDetailsService.java` line 25.

3. -> `UserRepository.findByEmail(email)` — `repository/UserRepository.java` line 10.

4. -> Hibernate/JDBC -> `SELECT * FROM users WHERE email = ?` on `helpdesk_db`.

5. Result wrapped in `CustomUserDetails` (`security/CustomUserDetails.java`); Spring Security BCrypt-verifies the password in memory.

6. On success -> `RoleBasedAuthenticationSuccessHandler.onAuthenticationSuccess()` (`security/RoleBasedAuthenticationSuccessHandler.java` line 19) redirects to `/student/dashboard`.

7. `StudentDashboardController.dashboard()` (`controller/StudentDashboardController.java` line 24) then calls `ticketService.myTickets(id)` -> `TicketRepository.findByStudentIdOrderByCreatedAtDesc` -> `SELECT * FROM tickets WHERE student_id = ? ORDER BY created_at DESC` — plus `registrationService.openSemesters()` -> `SELECT * FROM semesters WHERE registration_open = 1`.

4.2 Staff Creates a KB Article

1. Browser -> `POST /staff/kb` with the article form — `controller/StaffKbController.java` line 43 (`save()`).

2. -> `KnowledgeBaseService.saveArticle(form, principal.getUser())` (interface: `service/KnowledgeBaseService.java` line 24).

3. -> `KnowledgeBaseServiceImpl.saveArticle()` — `service/impl/KnowledgeBaseServiceImpl.java` lines 68–84.

4. -> `KbCategoryRepository.findById(form.getCategoryId())` — `SELECT * FROM kb_categories WHERE id = ?`.

5. Builds a new `KbArticle` (author = current staff user, `viewCount=0`).

6. -> `KbArticleRepository.save(article)` — `repository/KbArticleRepository.java` (inherited from `JpaRepository`) -> `INSERT INTO kb_articles (title, content, category_id, author_id, status, view_count, created_at) VALUES (...)`.

7. Redirect to `/staff/kb` -> `StaffKbController.list()` line 25 -> `KbArticleRepository.findAllByOrderByCreatedAtDesc()` -> `SELECT * FROM kb_articles ORDER BY created_at DESC` — new article now visible.

4.3 Student Submits a Support Ticket (with routing + notification side effects)

1. Browser -> `POST /student/tickets` — `controller/TicketController.java` line 45 (`create()`).

2. -> `TicketService.createTicket(student, form)` (interface `service/TicketService.java` line 12).

3. -> `TicketServiceImpl.createTicket()` — `service/impl/TicketServiceImpl.java` lines 38–55.

4. -> `TicketCategoryRepository.findById(form.getCategoryId())` — `SELECT * FROM ticket_categories WHERE id = ?` (this category row already carries its `pool_id`, set up beforehand via `AdminConfigController`/`AdminConfigServiceImpl`, Section 3.9).

5. -> `TicketRepository.save(ticket)` — `INSERT INTO tickets (student_id, category_id, subject, description, status, priority, ticket_ref, created_at) VALUES (...)`; SQL Server returns the new identity `id`.

6. Service sets `ticketRef = "TCK-%06d"` on the managed entity -> dirty-checked `UPDATE tickets SET ticket_ref = ? WHERE id = ?` at transaction commit (`@Transactional` on `createTicket`).

7. Redirect to `/student/tickets/{id}` -> `TicketController.detail()` line 57 -> `TicketService.getById(id)` (`SELECT * FROM tickets WHERE id = ?`) + `TicketService.commentsFor(id)` (`SELECT * FROM ticket_comments WHERE ticket_id = ? ORDER BY created_at ASC`).

8. **Downstream side effect (later, when staff act on it):** `StaffTicketController.assign()` -> `TicketServiceImpl.assignToSelf()` sets `assignedStaff`/`status=IN_PROGRESS` (dirty-checked `UPDATE tickets`) and calls `notifyStudent()` -> `NotificationService.notify()` -> `NotificationRepository.save()` -> `INSERT INTO notifications (recipient_id, type, message, related_entity_type, related_entity_id, is_read, created_at) VALUES (...)` — one user action (assigning a ticket) triggers **two** table writes (`tickets` + `notifications`). The staff member who can even see this ticket to assign it was itself determined by `TicketRepository.findQueueForStaff()`'s JPQL query joining `tickets.category.pool` against `JobPool.staffMembers` (i.e. the routing configuration from Section 3.9).

5. Seeing This Live in SQL Server Management Studio (SSMS)

The app connects as SQL login `helpdesk_user` to database `helpdesk_db` (from `application.properties` lines 12–14 and `SQLQuery2.sql` line 1–2).

1. Open **SQL Server Management Studio**.

2. In the "Connect to Server" dialog:

- Server name: `localhost,1433` (default instance, matching `spring.datasource.url`) — or `localhost\SQLEXPRESS` if using a named instance (per the comment on `application.properties` lines 9–11; the SQL Server Browser service must be running for a named instance).
- Authentication: **SQL Server Authentication**.
- Login: `helpdesk_user`, Password: `YourStrongPassword123!` (or whatever `DB_PASSWORD` was set to).

3. In **Object Explorer**, expand **Databases -> helpdesk_db -> Tables**. You should see (all created automatically by Hibernate's `ddl-auto=update`, no manual `CREATE TABLE` ever written):

`dbo.users`, `dbo.tickets`, `dbo.ticket_comments`, `dbo.ticket_attachments`, `dbo.ticket_categories`, `dbo.routing_rules`, `dbo.job_pools`, `dbo.pool_staff` (join table), `dbo.kb_articles`, `dbo.kb_categories`, `dbo.lost_found_items`, `dbo.lost_found_claims`, `dbo.semesters`, `dbo.courses`, `dbo.course_offerings`, `dbo.registrations`, `dbo.registration_courses` (join table), `dbo.notifications`, `dbo.feedback`, `dbo.audit_logs`.

4. Right-click any table (e.g. `dbo.tickets`) -> **Select Top 1000 Rows** to see its live data in a query results grid.

5. **To watch a row appear/change in real time for the demo:**

• Run the Spring Boot app (`mvn spring-boot:run`); watch the console — `spring.jpa.show-sql=true` (line 21) prints every SQL statement Hibernate sends, so you can literally show the lecturer the `INSERT INTO tickets (...)` line appearing the instant you submit the ticket form in the browser.

• In the browser, log in as a student and submit a new ticket (Section 3.2 / 4.3), or as staff create a KB article (Section 3.3 / 4.2), etc.

• Back in SSMS, either re-run the same **Select Top 1000 Rows** query (click **Execute** / F5 again — SSMS does not auto-refresh a result grid), or right-click the table in Object Explorer -> **Refresh** to confirm the row/column count updated, then re-run the select.

• This is the cleanest live demonstration that "the code talks to the database": browser action -> app console shows the generated SQL -> SSMS query re-run shows the new/changed row.

6. Demo/Seed Data Note

`config/DemoDataSeeder.java` implements `CommandLineRunner`, so its `run()` method executes automatically once on every application startup, gated by `app.seed-demo-data` (`application.properties` line 56, default `true` via `${SEED_DEMO_DATA:true}`).

What it seeds (only if not already present — each step checks `repository.count() > 0` first, so re-running the app doesn't duplicate data):

1. **Demo accounts** — `admin@helpdesk.university` / `Admin@123` (ADMIN) and `staff@helpdesk.university` / `Staff@123` (STAFF), via `seedUserIfMissing()` -> `userRepository.findByEmail(...).orElseGet(() -> ... userRepository.save(user))`. The class Javadoc explains *why* this is necessary: there is no public signup path for Staff/Admin (UC-02 — only an existing Admin can create one), so without this seeder nobody could ever log in as Staff/Admin on a fresh database.

2. **Ticket routing baseline** — one `JobPool` ("General Support"), three `TicketCategory` rows (IT/Academic/Facilities) all pointed at that pool, and one `RoutingRule` per category × priority combination.

3. **Knowledge base baseline** — one `KbCategory` ("General") and 3 published `KbArticle` rows (password reset, lost item reporting, ticket status checking).

4. **Semester catalog baseline** — one open `Semester`, two `Course` rows (SE2030, IT1010), and a `CourseOffering` for each in that semester.

To turn seeding off (recommended "before final submission/demo day" per the code comment), set the environment variable `SEED_DEMO_DATA=false` (or edit the property default). Everything the seeder creates can also just be edited/deleted afterwards through the normal admin screens, since it goes through the exact same repository/service methods as any real admin action.